Local Optimization

CS 1622

Jarrett Billingsley

Class Announcements

- buhhhhhhh??????

Optimization

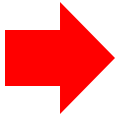
What is it?

- **optimization** is the name for any technique which:
  - **rewrites** part of a program…
  - in order to **improve resource utilization…**
  - *without* changing its **behavior.**
- "resource utilization" can mean many things, such as:
  - **time** (how quickly the code executes)
  - **code size** (how many bytes the code takes up)
  - **memory size** (how many bytes the *data/variables* take up)
  - **power** (how many watts the code uses when executed)
  - **registers** (how many/what kinds of registers are used)
- some of these goals are even contradictory!
  - so compilers let you control which optimizations are performed.

We do it all the time

- many of the optimizations we'll discuss are just **automated** versions of things human programmers do.

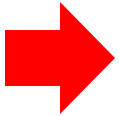
```
func1(some.long + code[i]);
func2(some.long + code[i]);
```



```
let v = some.long + code[i];
func1(v);
func2(v);
```

this technique is called **common subexpression elimination (CSE).**

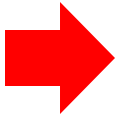
```
let x = 1024 * 4;
```



```
let x = 4096;
```

this is **constant folding.**

```
let x = pow(y, 2);
```



```
let x = y * y;
```

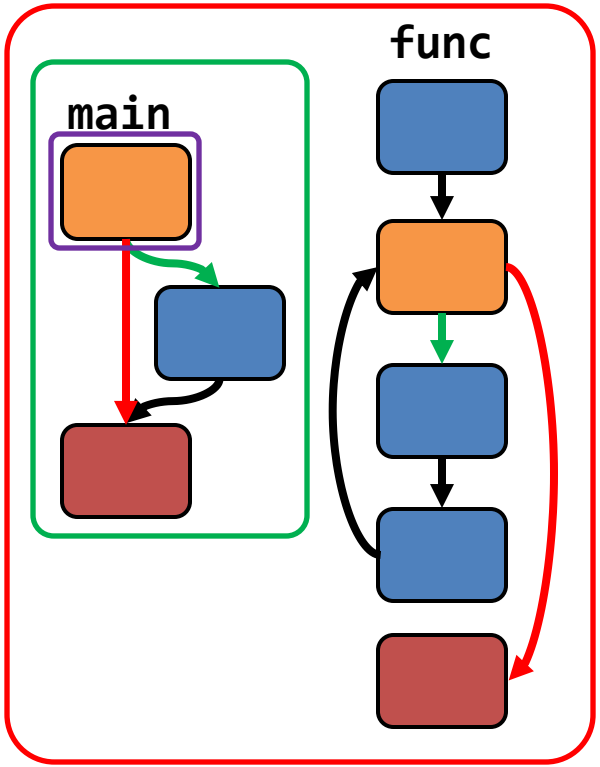
this is **strength reduction:** using a simpler, faster operation that gives the same answer as a more complex one.

Optimization scope

- we'll be doing optimizations on our IR, rather than the AST.

local optimizations work on code within **a single BB.**

global optimizations work on ***one function's*** whole CFG. (I know, "global" sounds like it should be the whole program but it's not)



interprocedural optimizations work on ***all functions in the program.***

as you might imagine, **local** is the simplest; **global** is more complex; and **interprocedural** is the most complex.

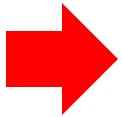
Dead code elimination (DCE)

- a common technique in Java is to use **static final** variables to enable or disable pieces of code. in those cases, the condition is a constant, and one or more control paths become "dead code."

```
static final boolean DEBUG = false;

public int getWhatever() {
  if(DEBUG)
    System.err.println("here");

  return this.whatever;
}
```



```
public int getWhatever() {
  return this.whatever;
}
```

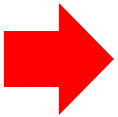
this is a **global** optimization, as it operates on the CFG of the method. some BBs were removed from this method.

Function (or method) inlining

- a very common interprocedural optimization is **function inlining:** eliminating a call by copying the code of the callee into the caller.

```
public void printWhatever() {
  int w = this.getWhatever();
  System.out.println(w);
}

public int getWhatever() {
  return this.whatever;
}
```



```
public void printWhatever() {
  int w = this.whatever;
  System.out.println(w);
}
```

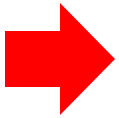
think about all the work this is saving – no more passing arguments, doing the function prologue and epilogue, messing with the stack…

but knowing when it's **okay to do this,** and knowing whether doing this will **save time or waste time** is really, really complicated.

Careful now…

- remember the "without changing its behavior" part of the definition?

```
func1(problem(1));
func2(problem(1));
```



```
let v = problem(1);
func1(v);
func2(v);
```

what if **problem** were defined like this:

```
fn problem(x: int): int {
    print_s("problem: ");
    println_i(x);
    return x;
}
```

now the original and "optimized" versions **do different things!**

when performing optimizations, we must respect the **semantics** and **evaluation rules** of the source language, and we **cannot ignore side effects.**

To optimize, or not to optimize?

- optimization might sound like a no-brainer. why *not* do it? well…

they can **slow down compilation, *a lot*.**

```
Finished [release] target(s) in 1859.3s
```

they can be **difficult to implement correctly.**

```
$ ./hello
Hello, world!
$ ./hello_optimized
Segmentation fault
```

they may **only give a small improvement.**

```
$ ./bench
18.93s
$ ./bench_optimized
18.11s
```

they can make it **harder to debug a running program.**

```
$ gdb bench_optimized
(gdb) b main
Function "main" not defined.
(gdb) _
```

or some combination of all of these!

Peephole Optimizations

Starting small

- the simplest kinds of optimizations are **peephole optimizations:** they work on the level of one or two instructions at a time.
- despite their simplicity, **they are the foundation** of all the more complicated kinds of optimizations.

```
a = 5 == 5
```



```
a = true
```

doing peephole optimizations…

```
a = true
if a then bb1
     else bb2
```



```
if true then bb1
        else bb2
```

…can open up opportunities for larger local optimizations…

```
if true then bb1
        else bb2
```



```
goto bb1
<delete bb2>
```

…which are the basis for global optimizations.

Operations vs. Moves (assignments)

- each instruction in our IR looks (basically) like one of these two:

```
x = y + z
a = b
```

- the first is an **operation:** it requires computation to complete.
- the second is a **move:** copying a value from place to place.
  - but, **a lot of moves are unnecessary** and *could* be eliminated.
- furthermore, if we can somehow **convert operations into moves…**
  - we can eliminate entire steps of the program.
- all the local optimizations basically follow these principles:
  - make operations **simpler than what was written;**
  - **turn operations into moves** if possible; and then
  - **eliminate the moves.**
- and then **repeat!**

Constant Folding

- this one is easy: **if you see arithmetic being done on constants, replace the arithmetic with the result of the operation.**

```
a = 640 * 480        →    a = 307200
b = not true         →    b = false
c = "hi" + "bye"     →    c = "hibye"
```

this turns each operation into a move. nice! but what about:

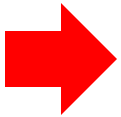
```
$t1 = 640 * 480      →    $t1 = 307200
x   = $t1 * 32       →    x   = $t1 * 32
```

clearly there's more work to be done, but we'll come back to this.

Danger ahead

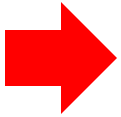
- some mathematical operations can cause… problems.

```
a = 1000 / 0
```



uh oh. don't try to simplify this or you'll **crash the compiler!**

```
a = 1000000000 * 5
```



that doesn't fit in a 32-bit int…

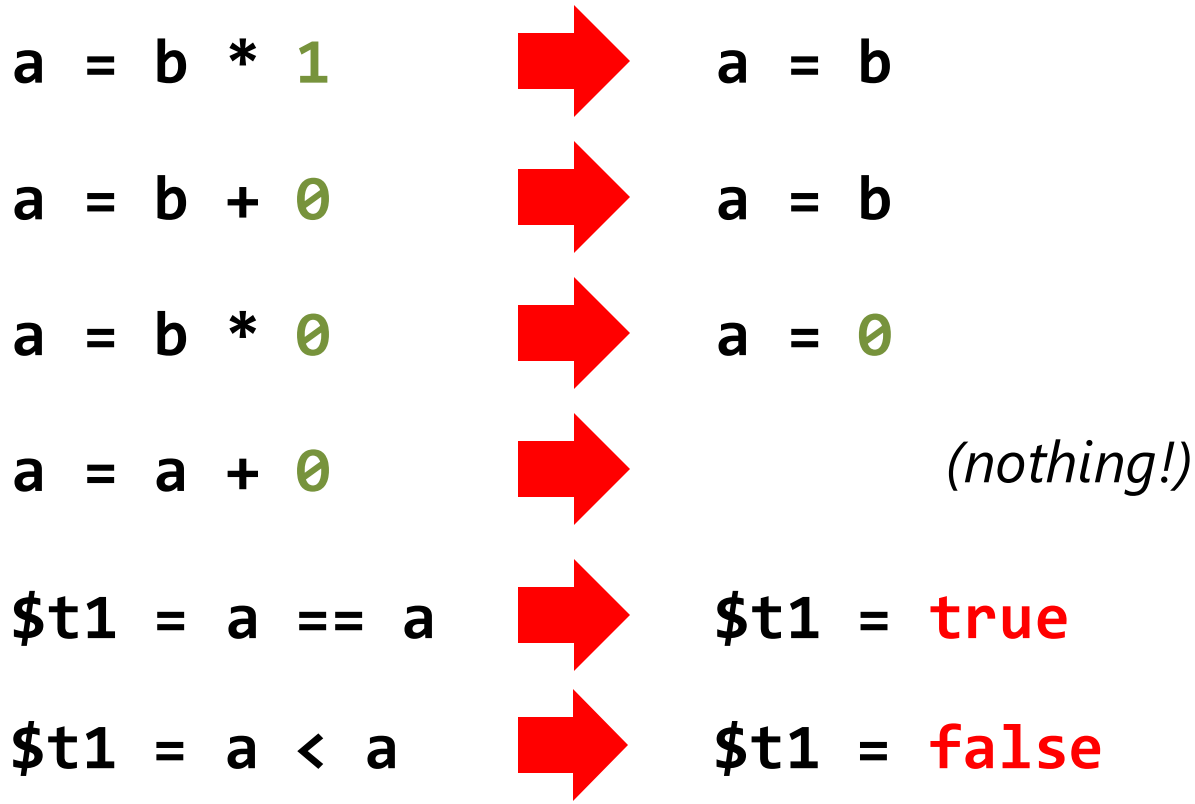
confusingly, these **may or may not** be errors in the code, so we probably shouldn't give an error here; just give up instead.

due to interactions with control flow analysis, these *may not even get executed at runtime*.

last, unless we're explicitly looking for errors, we really should just let the code through. **we can't catch every mistake.** it's never wrong to *leave code the way the programmer wrote it*.

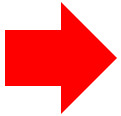
Algebraic Simplification

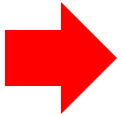
- this uses **algebraic laws** to simplify or even eliminate operations.



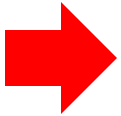
Strength Reduction

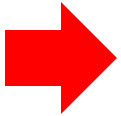
- **strength reduction** might not reduce complexity, but it can improve performance by using **cheaper operations** to do the same task.

```
a = b * 2
```

```
a = b + b
```

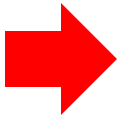
```
a = pow(b, 2)
```

```
a = b * b
```

(many compilers "know" about standard library math functions and can optimize them out.)

```
a = b * 8
```

```
a = b << 3
```

```
a = b * 9
```

```
a = b << 3
a = a + b
```

two instructions may still be faster than a single multiplication!

```
a = b % 8
```

```
a = b & 7
```

Optimization: not just for IR

- peephole optimizations can be done *on the target code too.*
- here are some Silly Sequences my code generator is producing:

```
addi sp, sp, -4
addi sp, sp, -4
```

shows up in nested calls like ***f(g(x))***

```
li  s2, 5
add s1, s1, s2
```

adding a constant to a variable, ***x + 5***

```
jal  func
move s0, v0
sw   s0, -16(fp)
```

assigning a return value to a variable

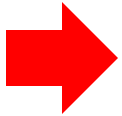
```
jal  func
move s0, v0
move v0, s0
```

returning the value that a call returned

Make it better, do it faster

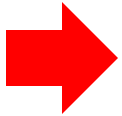
- a final pass after codegen can go through and replace these silly sequences with simpler equivalent sequences.

```
addi sp, sp, -4
addi sp, sp, -4
```



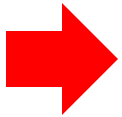
```
addi sp, sp, -8
```

```
li   s2, 5
add  s1, s1, s2
```



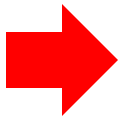
```
addi s1, s1, 5
```

```
jal  func
move s0, v0
sw   s0, -16(fp)
```



```
jal  func
sw   v0, -16(fp)
```

```
jal  func
move s0, v0
move v0, s0
```



```
jal  func
```

Single Static Assignment Form (SSA)

Hitting a wall

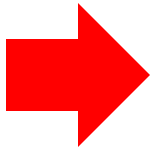
- let's apply all the optimizations we've seen to this bit of code.

```
x = 10 * 16
println_i(x)
x = x * 4
println_i(x)
x = x * 1
println_i(x)
$t0 = x
return
```

constant folding...

strength reduction...

algebraic simplification...



```
x = 160
println_i(x)
x = x << 2
println_i(x)


println_i(x)
$t0 = x
return
```

kind of leaves something to be desired, no?

think about it: **do we really even *need* this x variable?**

well, to make things easier on ourselves, first we're going to convert the code into a **single assignment form.**

I AM LEAVING OUT A LOT OF DETAILS HERE

- **single static assignment (SSA) form** rewrites the code so that each location is only assigned *once*, and is never *reassigned*.
- **this does not "optimize" the code at all,** but it *does* make several other optimizations *much* easier to perform.

```
x = 10 * 16
println_i(x)
x = x * 4
println_i(x)
x = x * 1
println_i(x)
$t0 = x
return
```




```
x1 = 10 * 16
println_i(x1)
x2 = x1 * 4
println_i(x2)
x3 = x2 * 1
println_i(x3)
$t0 = x3
return
```

each time **x is assigned,** we replace it with a new, unique variable.

references to x are replaced with the "most recent version" of **x**.

Not a requirement

- SSA is a super useful form for optimization *buuuuuuuuuut…*
  - it has to be done on the whole CFG, which is **complicated.**
  - consider a ~~for~~ loop. how do you represent the counter variable in this form at all?? (heheheheh hahah hohohoho ϕ)
- then, once you're done optimizing, **you have to convert back out of SSA** to do the final codegen.
  - so it's a bit of a mixed bag, depending on what kinds of optimizations you want to do.
- it's not *required* for all optimizations, it just simplifies a lot of them.
  - the other local optimizations we'll talk about can be implemented with or without SSA, but the algorithms for detecting and applying them are more complicated without it.

More advanced local optimizations

Copy Propagation

- **if the code is in SSA form,** we "unlock" this optimization:
- if we see a move **x = y**, then we can replace all uses of **x** with **y**.

```
x = a                x = a
b = x + c     ⇒      b = a + c

x = 10               x = 10
a = x + 5     ⇒      a = 10 + 5
```

two things to notice here:

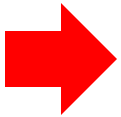
in the second example, we've produced something that can be **further optimized** to **a = 15**!

in both examples, **x is no longer used anywhere.** so what do we do with it?

Dead store elimination

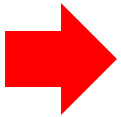
- if a variable is assigned a value, **and that value is never read,** then the assignment is a **dead store** and can be **removed.**
  - *...as long as **the assignment has no side effects!***
- so continuing with the improved code from last slide:

```
x = a
b = a + c
```



```
b = a + c
```

```
x = 10
a = 10 + 5
```



```
a = 10 + 5
```

```
x = f()
$t0 = 5
return
```

but:

we can't remove the assignment to **x**, because **f()** may have side effects.

Common subexpression elimination (CSE)

- **if the code is in SSA form...**
  - and two variable assignments have the **same rhs...**
  - **and the rhs has no side effects...**
- then the second variable assignment can be changed to a **move from the first variable.**

```
x = a + c                x = a + c
println_i(x)             println_i(x)
y = a + c        ➡       y = x
println_i(y)             println_i(y)
```

because we're using SSA, the variables on the RHS are guaranteed not to change value between the assignments.

(if we weren't using SSA, we'd have to check that.)

Teamwork makes the dream work

- each local optimization **only does a little work.**
- but every time you run one, it can make it possible for another one to do a little more work…



a copy propagation makes a constant folding possible;

which makes another copy propagation possible, followed by a dead store elimination;

around and around until the code is as "simple" as we can make it.

How the compiler does it

- it just does a simple round-robin scheme:

```
do {
    const_fold(bb);
    strength_reduce(bb);
    algebraic_simpl(bb);
    copy_propagate(bb);
    dead_store_elim(bb);
} while(the bb changed);
```

each optimization is so simple and quick that you just... keep doing them until they don't do anything anymore.

it is literally more work to "check if the optimization is possible" than it is to just try to do it

if this makes you feel uneasy (couldn't it get stuck in an infinite loop??), I'm with you! but if it works, it works.

(the compiler might stop after some *n* iterations anyway).

An example

- let's take that code from before and optimize it real good.

```
x1 = 10 * 16
println_i(x1)
x2 = x1 * 4
println_i(x2)
x3 = x2 * 1
println_i(x3)
$t0 = x3
return
```

```
x1 = 160
println_i(x1)
x2 = x1 << 2
println_i(x2)
x3 = x2
println_i(x3)
$t0 = x3
return
```

```
x1 = 160
println_i(160)
x2 = 160 << 2
println_i(x2)
x3 = x2
println_i(x2)
$t0 = x2
return
```

```
println_i(160)
x2 = 160 << 2
println_i(x2)
println_i(x2)
$t0 = x2
return
```

```
println_i(160)
x2 = 640
println_i(x2)
println_i(x2)
$t0 = x2
return
```

```
println_i(160)
x2 = 640
println_i(640)
println_i(640)
$t0 = 640
return
```

```
println_i(160)
println_i(640)
println_i(640)
$t0 = 640
return
```

whew!

A bigger example of CSE

- even larger common subexpressions can be eliminated.

```
func_1(a.x + b.y - c);
func_2(a.x + b.y - c);
```

```
$t1 = a.x + b.y
$t2 = $t1 - c
func_1($t2)
$t3 = a.x + b.y
$t4 = $t3 - c
func_2($t4)
```

```
$t1 = a.x + b.y
$t2 = $t1 - c
func_1($t2)
$t3 = $t1
$t4 = $t3 - c
func_2($t4)
```

```
$t1 = a.x + b.y
$t2 = $t1 - c
func_1($t2)
$t3 = $t1
$t4 = $t1 - c
func_2($t4)
```

```
$t1 = a.x + b.y
$t2 = $t1 - c
func_1($t2)
$t4 = $t1 - c
func_2($t4)
```

```
$t1 = a.x + b.y
$t2 = $t1 - c
func_1($t2)
$t4 = $t2
func_2($t4)
```

```
$t1 = a.x + b.y
$t2 = $t1 - c
func_1($t2)
$t4 = $t2
func_2($t2)
```

```
$t1 = a.x + b.y
$t2 = $t1 - c
func_1($t2)
func_2($t2)
```

